```
{
   "identifier": { "entityType": "ACME::Document", "entityId": "q3-
plan" },
   "attributes": {
     "owner": { "entityIdentifier": { "entityType": "ACME::Employe
e",
                                      "entityId": "alice" } },
        "classification": { "string": "confidential" },
        "delegatable": { "boolean": true },
        "employee_readers_team": {
          "entityIdentifier": { "entityType": "ACME::Team",
                                "entityId": "doc-q3-employee-reader
s" }
        },
        "customer_readers_team": {
          "entityIdentifier": { "entityType": "ACME::Team",
                                "entityId": "custco-readers" }
        }
      }
    }
```

By defining both an employee readers team and a customer readers team for each document, ACME ensures that only the designated groups can access it. In this case, there is a dedicated `doc-q3-employee-readers` team but only a general `custco` readers team. This suggests that ACME creates a unique employee readers team for each document while reusing a single readers team for each customer. If ACME chose to structure the relationships differently, the policies would not need to change—only how the entities are related.

A.5.5 Using the entity definitions

At ACME, the identity team doesn’t manage static JSON files of employees, customers, and documents. Instead, they manage a live relationship graph of entities and edges that acts as the single source of truth. Each authorization decision pulls just the slice of that graph that’s relevant to the request. In AVP terms, this relationship graph is called the *entity store*.